

Modbus RTU

Example

Note:

uModbus doesn't support all the functions defined for Modbus RTU. It currently supports the following functions:

- 01: Read Coils
- 02: Read Discrete Inputs
- 03: Read Holding Registers
- 04: Read Input Registers
- 05: Write Single Coil
- 06: Write Single Register
- 15: Write Multiple Coils
- 16: Write Multiple Registers

```
#!/usr/bin/env python
# scripts/example/simple_rtu_client.py
import fcntl
import struct
from serial import Serial, PARITY_NONE

from umodbus.client.serial import rtu

def get_serial_port():
    """ Return serial.Serial instance, ready to use for RS485. """
    port = Serial(port='/dev/ttyS1', baudrate=9600, parity=PARITY_NONE,
                  stopbits=1, bytesize=8, timeout=1)

    fh = port.fileno()

    # A struct with configuration for serial port.
    serial_rs485 = struct.pack('hhhhhhh', 1, 0, 0, 0, 0, 0, 0)
    fcntl.ioctl(fh, 0x542F, serial_rs485)

    return port

serial_port = get_serial_port()

# Returns a message or Application Data Unit (ADU) specific for doing
# Modbus RTU.
message = rtu.write_multiple_coils(slave_id=1, address=1, values=[1, 0, 1, 1])

# Response depends on Modbus function code. This particular returns the
# amount of coils written, in this case it is.
response = rtu.send_message(message, serial_port)

serial_port.close()
```

`umodbus.client.serial.rtu.send_message(adu, serial_port)` [\[source\]](#)

Send ADU over serial to to server and return parsed response.

Parameters: • *adu* – Request ADU.
• *sock* – Serial port instance.

Returns: Parsed response from server.

`umodbus.client.serial.rtu.parse_response_adu(resp_adu, req_adu=None)` [\[source\]](#)

Parse response ADU and return response data. Some functions require request ADU to fully understand request ADU.

Parameters: • *resp_adu* – Resonse ADU.
• *req_adu* – Request ADU, default None.

Returns: Response data.

`umodbus.client.serial.rtu.read_coils(slave_id, starting_address, quantity)` [\[source\]](#)

Return ADU for Modbus function code 01: Read Coils.

Parameters: *slave_id* – Number of slave.
Returns: Byte array with ADU.

`umodbus.client.serial.rtu.read_discrete_inputs(slave_id, starting_address, quantity)` [\[source\]](#)

Return ADU for Modbus function code 02: Read Discrete Inputs.

Parameters: *slave_id* – Number of slave.
Returns: Byte array with ADU.

`umodbus.client.serial.rtu.read_holding_registers(slave_id, starting_address, quantity)` [\[source\]](#)

Return ADU for Modbus function code 03: Read Holding Registers.

Parameters: *slave_id* – Number of slave.
Returns: Byte array with ADU.

`umodbus.client.serial.rtu.read_input_registers(slave_id, starting_address, quantity)` [\[source\]](#)

Return ADU for Modbus function code 04: Read Input Registers.

Parameters: *slave_id* – Number of slave.
Returns: Byte array with ADU.

`umodbus.client.serial.rtu.write_single_coil(slave_id, address, value)` [\[source\]](#)

Return ADU for Modbus function code 05: Write Single Coil.

Parameters: *slave_id* – Number of slave.
Returns: Byte array with ADU.

`umodbus.client.serial.rtu.write_single_register(slave_id, address, value)` [\[source\]](#)

Return ADU for Modbus function code 06: Write Single Register.

Parameters: *slave_id* – Number of slave.
Returns: Byte array with ADU.

`umodbus.client.serial.rtu.write_multiple_coils(slave_id, starting_address, values)` [\[source\]](#)

Return ADU for Modbus function code 15: Write Multiple Coils.

Parameters: *slave_id* – Number of slave.
Returns: Byte array with ADU.

`umodbus.client.serial.rtu.write_multiple_registers(slave_id, starting_address, values)`

[\[source\]](#)

Return ADU for Modbus function code 16: Write Multiple Registers.

Parameters: `slave_id` – Number of slave.

Returns: Byte array with ADU.